

Level 0 Report: Dataset Collection & Chunking Strategy

Route Graduation Project
Intelligent RAG System

Executive Summary

This report documents the data collection and chunking strategy implemented for Level 0 of the Intelligent RAG System project. The dataset comprises 1,000 book records scraped from the Books to Scrape website, cleaned and preprocessed for consistency, with an effective chunking strategy designed to preserve semantic meaning. The dataset consists of book titles, prices, ratings, descriptions, and source URLs, structured as complete product records ready for embedding and retrieval in later project phases.

1. Dataset Description

1.1 Website and Source

- **Source:** Books to Scrape
- **URL:** <http://books.toscrape.com/>

Books to Scrape is a publicly accessible website designed specifically for practicing web scraping techniques. It contains a catalog of 1,000 books spread across 50 pages, with 20 books per page.

1.2 Source URLs

The scraping process covered pages 1 through 50:

```
http://books.toscrape.com/catalogue/page-{1-50}.html
```

Each book page URL follows the format:

```
http://books.toscrape.com/catalogue/[book-slug]_[book-id]/index.html
```

1.3 Type of Data Collected

The dataset consists of HTML pages containing product records with the following structure:

Field	Type	Description
Title	Text	Book title (string)
Price	Text	Price in GBP with £ symbol
Rating	Text	Star rating (One to Five)
Description	Text	Book description from product page
URL	Text	Source URL for reference

1.4 Data Structure

Each record follows this JSON structure:

```
{
  "title": "A Light in the Attic",
  "price": "£51.77",
  "rating": "Three",
  "description": "It's hard to imagine a world without A Light in the Attic...",
  "url": "http://books.toscrape.com/catalogue/a-light-in-the-attic_1000/index.html"
}
```

1.5 Number of Records Collected

Total: 1,000 book records

The collection process successfully scraped all books from:

- 50 pages total
- 20 books per page
- 1,000 complete records

1.6 Final Dataset Format

The dataset was saved in JSON format as `books_dataset.json` for:

- Easy loading and manipulation
- Compatibility with Pandas DataFrames
- Preserving nested data structure
- Human-readable storage

2. Data Collection Method

2.1 Tools and Libraries Used

Library	Version	Purpose
requests	Latest	HTTP requests for web pages
BeautifulSoup (bs4)	Latest	HTML parsing and data extraction
json	Built-in	JSON file handling
pandas	Latest	Data manipulation and cleaning
time	Built-in	Rate limiting and polite scraping

2.2 Scraping Implementation

Step 1: Base URL Configuration

```
BASE_URL = "http://books.toscrape.com/catalogue/page-{}.html"
HEADERS = {"User-Agent": "Mozilla/5.0"}
```

Step 2: Pagination Loop

- Iterated through pages 1 to 50
- Stopped when no products found on a page
- 0.2-second delay between requests for politeness

Step 3: Data Extraction Process

For each book on each page:

Extract Basic Information

- Title from <h3><a> tag's title attribute
- Price from <p class="price_color">
- Rating from <p class="star-rating [rating]"> class attribute

Navigate to Individual Book Page

- Constructed full URL from relative link
- Opened individual book page

Extract Description

- Located <div id="product_description">
- Extracted following <p> tag content
- Set "No description" as fallback

Store Data

- Appended record to books list
- Saved to JSON file after completion

2.3 Cleaning and Preprocessing Steps

A comprehensive cleaning function was applied to ensure data quality:

```
def clean_text(text):
    if text is None:
        return ""
    text = text.replace("\n", " ")      # Remove newline characters
    text = text.replace("\r", " ")      # Remove carriage returns
    text = " ".join(text.split())       # Normalize whitespace
    return text.strip()                 # Remove leading/trailing spaces
```

Applied to fields:

- Title
- Description
- Price
- Rating

Benefits of cleaning:

- Consistent formatting across all records
- Removed problematic escape sequences
- Normalized whitespace for embedding quality
- Clean string conversion for all fields

2.4 Challenges Faced During Data Collection

Challenge	Solution
Page Navigation	Used while loop with break condition when no products found
Missing Descriptions	Implemented fallback to "No description"
Relative URLs	Constructed full URLs by concatenating with base URL
Server Load	Added 0.2-second delay between requests
Encoding Issues	Used encoding='utf-8' in JSON dump
Inconsistent Structure	Used BeautifulSoup with try/except patterns

2.5 Verification Steps

After collection, the following verification steps were performed:

- All 1,000 records complete
- No missing fields
- Clean text formatting
- Valid JSON structure
- All URLs accessible

3. Chunking Strategy

3.1 Selected Chunking Method

Strategy: One book = one chunk

Each chunk contains:

- Book title
- Price
- Rating
- Description

3.2 Implementation

```
chunks = []
for index, row in df.iterrows():
    chunk = f"""
Title: {row['title']}
Price: {row['price']}
Rating: {row['rating']}
Description:
{row['description']}
"""
    chunks.append(chunk)
```

3.3 Reasoning Behind the Method

1. Natural Unit of Meaning

- Each book record represents a complete, independent product
- All related information (title, price, rating, description) belongs together
- Splitting would break the semantic coherence

2. Data Observation

- Book descriptions are relatively short (average 100-300 words)
- The description alone does not capture the book's context without metadata
- Users ask about complete books, not paragraphs

3. Retrieval Quality

- A complete book chunk provides rich context for Gemini
- Preserves relationships between book attributes
- Allows retrieval of complete information about a book

4. Query Patterns

- Users ask questions such as: "What books have a five-star rating?"
- Users ask: "Show me cheap books with good ratings"
- Both require complete book information, not partial chunks

3.4 Alternatives Considered

Alternative	Why Rejected
Sentence-level Chunking	Loses semantic meaning; separates related information
Paragraph-level Chunking	Splits title from description; damages retrieval quality
Description-only Chunking	Loses context about price and rating
Metadata-only Chunking	Loses important descriptive context
Overlapping Chunks	Unnecessary for independent records; adds redundancy

3.5 Weaknesses of the Chosen Strategy

1. Large Chunk Size

- Some chunks are long (~200-400 tokens)
- May exceed context window limits for very large datasets
- Mitigation: Book descriptions are naturally short; 1,000 chunks of moderate size

2. No Chunk Overlap

- No redundancy to capture information across boundaries
- Mitigation: Each record is independent; overlap unnecessary

3. Inconsistent Chunk Sizes

- Descriptions vary in length significantly
- Mitigation: Embedding models handle variable lengths well; no fixed size required

4. Duplicate Information

- Price appears in both metadata and chunk text
- Mitigation: Chunk text is used for embedding; metadata for filtering

3.6 Mitigation Strategies

Weakness	Mitigation
Large chunks	Descriptions are short; each chunk ~200-400 tokens
No overlap	Independent records; overlap would add noise
Variable length	Embedding models handle variable lengths naturally
Duplicate info	Metadata stored separately for filtering; chunk for retrieval

3.7 Metadata Strategy

Each chunk stores comprehensive metadata:

```
metadata = {
  "record_id": index,
  "title": row["title"],
  "price": row["price"],
  "rating": row["rating"],
  "source_url": row["url"]
}
```

The rationale for each metadata field is as follows:

Field	Purpose
record_id	Unique identifier for tracking
title	Quick display and filtering
price	Filtering by price range
rating	Filtering by rating
source_url	Traceability and source verification

3.8 Final Dataset Statistics

Metric	Value
Total Records	1,000
Chunks Created	1,000
Average Chunk Length	~300 words
Average Description Length	~150 words
Metadata Fields	5 per chunk
Total File Size	~1.2 MB (JSON)

4. Quality Assurance

4.1 Data Quality Checks

- Completeness: All 1,000 records have title, price, rating, description, and URL
- Consistency: All fields cleaned and normalized
- Encoding: UTF-8 encoding preserved special characters
- Structure: Valid JSON format maintained

4.2 Sample Record Verification

```
Title: A Light in the Attic  
Price: £51.77  
Rating: Three  
Description: It's hard to imagine a world without A Light in the Attic...  
URL: http://books.toscrape.com/catalogue/a-light-in-the-attic_1000/index.html
```

4.3 Chunk Quality Verification

- All chunks contain title, price, rating, and description
- No empty descriptions (fallback text exists)
- Proper formatting preserved
- Metadata accurately maps to chunk content

Conclusion

Level 0 has been successfully completed with the following achievements:

- Collected 1,000 high-quality book records from Books to Scrape
- Cleaned and preprocessed all data for consistency
- Designed and implemented an effective chunking strategy
- Created comprehensive metadata for each chunk
- Stored data in JSON format for easy loading in Level 1

The dataset is ready for:

- Embedding generation
- Vector storage
- Retrieval-Augmented Generation

The one-book-per-chunk strategy ensures semantic coherence and preserves the natural unit of meaning, setting a strong foundation for the RAG pipeline in Level 1.